

Homework — 2.1.5 Thinking Concurrently

Name: _____ Date: _ **Mark:** ____ / 20

OCR H446 — set after the 2.1.5 lesson.

Part A — This week's topic (~14 marks)

1. A 3-D animation studio must render the 200 individual frames of a short film. Each frame is calculated independently of the others.

(a) Explain why these frames are well suited to **concurrent (parallel) processing**, referring to data dependency. **[2]** (AO2)

(b) State **one** hardware feature that would let the render finish genuinely faster. **[1]** (AO1)

2. A spreadsheet app, when the user clicks "Save", also runs a long spell-check in the background so the user can keep typing.

Explain **one** benefit of running the spell-check **concurrently** with editing. **[2]** (AO2)

3. Two members of staff use the same warehouse system at the same instant. Both read "stock = 1" for the last item and both confirm a sale, so the item is sold twice.

(a) Name the problem this illustrates. **[1]** (AO1)

(b) Explain **why** it happened, referring to shared data and timing. [2] (AO2)

4. A student claims: "Running my program's tasks concurrently will *a/ways* make it finish faster." Give **two** reasons why this is not necessarily true. [2] (AO2)

5. A holiday-booking site, when a customer books a trip, must: (a) charge the customer's card, (b) reserve the flight seat, (c) reserve the hotel room, and (d) send a confirmation email.

Using *thinking concurrently*, state which of these tasks could run at the **same time** and which must be **sequenced**, then discuss **one** benefit and **one** trade-off of running tasks concurrently here. [4] (AO2/AO3)

Part B — Retrieval: keep it fresh (~6 marks)

6. (2.1.4) A game ends a player's turn when they have **either** run out of moves **or** scored 100 points. Write a suitable Boolean **condition** using a logical operator. [2] (AO2)

7. (1.4.3) Complete the truth table for $Q = A \text{ AND } (\text{NOT } B)$. [2] (AO2)

A	B	Q
0	0	
0	1	
1	0	
1	1	

8. (1.4.2) State **one** difference between a **static** and a **dynamic** data structure. [2] (AO1)
